

Pagination & Sorting in Java

Build scalable APIs with Spring Boot
(Essential for real-world backend systems)



The Problem

✗ Without Pagination

```
// Loading 100,000 records
List<User> users = userRepo.findAll();
// ✗ High memory usage
// ✗ Slow response time
```

✓ With Pagination

```
// Loading only 20 records
Page<User> users =
    userRepo.findAll(
        PageRequest.of(0, 20));
// ✓ Fast & efficient
```

✦ No pagination = poor scalability

Repository Setup

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    // Built-in pagination
    Page<User> findAll(Pageable pageable);

    // Pagination + filtering
    Page<User> findByStatus(
        String status,
        Pageable pageable);
}
```

✦ Spring Data handles LIMIT, OFFSET, and count queries automatically

Pageable Object

```
// Page 0, size 10
Pageable pageable =
    PageRequest.of(0, 10);

// With sorting
Pageable pageable =
    PageRequest.of(0, 10,
        Sort.by("name").descending());

// Multiple sort fields
Sort sort = Sort.by(
    Sort.Order.asc("lastName"),
    Sort.Order.desc("age"));

Pageable pageable =
    PageRequest.of(0, 10, sort);
```

📌 Page index starts from 0

REST Controller (Clean Way)

```
// GET /users?page=0&size=20&sort=name,asc

@GetMapping("/users")
public Page<UserResponse> getUsers(
    @PageableDefault(
        size = 10,
        sort = "id"
    ) Pageable pageable) {

    return userRepository
        .findAll(pageable)
        .map(UserMapper::toDto);
}
```

- ✓ Cleaner than manual @RequestParam
- ✓ Interview-approved

Page Response

```
Page<User> page = userRepo.findAll(pageable);
```

Useful methods

```
page.getContent();           // Current records
page.getTotalElements();     // Total rows
page.getTotalPages();        // Total pages
page.getNumber();           // Current page
page.hasNext();              // Next page?
page.isLast();               // Last page?
```

 Perfect for frontend pagination UI

Best Practices

✓ DO

- Index sortable columns
- Set default sorting
- Limit max page size (≤ 100)
- Return Page<DTO>
- Validate page & size

✗ DON'T

- Load all records
- Allow unlimited page size
- Expose entities
- Use deep pagination blindly

Pagination is NOT optional

- ✓ Use PageRequest.of(page, size)
- ✓ Combine pagination + sorting
- ✓ Prefer Pageable in controllers
- ✓ Return metadata to clients
- ✓ Index your sort columns

| 🖱️ No pagination means your API works... until it doesn't.

Pagination and sorting protect performance and scalability.

IF YOU FIND THIS HELPFUL, LIKE AND
SHARE IT WITH YOUR FRIENDS

ALAN BIJU | [@alanbiju98](#)